

Comparing the new QRC with the old QRC

Jacob L. Cybulski

Summary: The old model performs better than the new model, primarily because it is not using its long-term memory (reservoir) at all and relies only on the short-term memory (windows). The new model utilises both types of memory as per the algorithm, combining them and injecting some regularisation / weighing the short and long term memories. It seems that this strategy is not as effective as using the short-term (windows) memory only. However, the new model effectively demonstrates how the QRC model works.

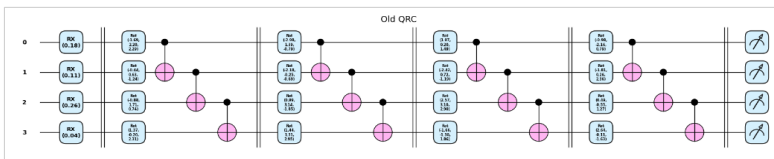
The old model is encoding time series windows with one Rx per qubit, followed by rotational (U) + entangling (linear) blocks:

```
@qml.qnode(dev)
def quantum_feature_map(inputs, weights, biases, n_layers=1, n_qubits=4):

    # Encode the classical input into the quantum state vector
    for i in range(n_qubits):
        total_angle = inputs[i] + biases[i]
        qml.RX(total_angle, wires=i)
    qml.Barrier()

    # Reservoir dynamics (fixed weights)
    for layer in range(n_layers):
        for i in range(n_qubits):
            qml.Rot(*weights[layer, i], wires=i)
        for i in range(n_qubits - 1):
            qml.CNOT(wires=[i, i + 1])
        qml.Barrier()

    # Return quantum features (observables)
    return [qml.expval(qml.PauliX(i)) for i in range(n_qubits)] + \
           [qml.expval(qml.PauliY(i)) for i in range(n_qubits)] + \
           [qml.expval(qml.PauliZ(i)) for i in range(n_qubits)]
```



First, as data is encoded in Rx form, the ansatz simply performs a linear transformation of input into output. More importantly, the main problem with the model is that the windows elements are treated as completely independent variables (no correlation between them). Also, as they are drawn from the time series, their values are incredibly close and they are highly correlated. As a result, they are clustered in the Hilbert space and highly homogenised, thus scrambling the spatialised output of the quantum feature map, while remaining highly redundant as compared with data on input. Consequently, the resulting high-dimensional output not very useful when combined with the reservoir. This happens until the leakage rate allows relying less and less on the reservoir memory.

The new model is encoding time series windows with ZZ feature map, followed by rotational (U) + reuploading (Rx per qubit) + entangling (circular) blocks:

```
@qml.qnode(dev)
def quantum_feature_map_v2(inputs, weights, biases, n_layers=1, n_qubits=4):

    # Encode the classical input into the quantum state vector
    zz_feature_map(inputs)
    qml.Barrier()

    # Reservoir dynamics (fixed weights)
    for layer in range(n_layers):
        # Rotational layer
        for i in range(n_qubits):
            qml.Rot(*weights[layer, i], wires=i)
        # Entanglement layer
        for i in range(n_qubits - 1):
            qml.CNOT(wires=[i, i + 1])
        qml.CNOT(wires=[n_qubits-1, 0])
        qml.Barrier()
        # Reuploading layer
        if layer+1 < n_layers:
            for i in range(n_qubits):
                total_angle = inputs[i] + biases[i]
                qml.RX(total_angle, wires=i)
            qml.Barrier()

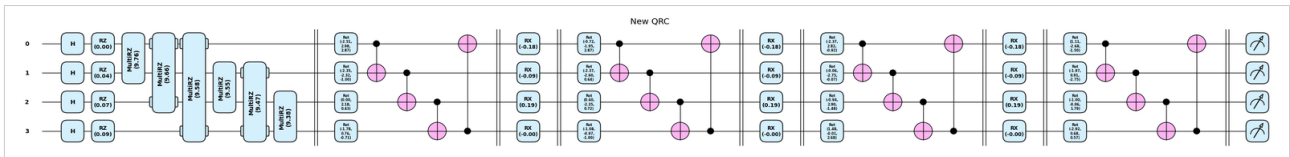
    # Return quantum features (observables)
    return [qml.expval(qml.PauliX(i)) for i in range(n_qubits)] + \
           [qml.expval(qml.PauliY(i)) for i in range(n_qubits)] + \
           [qml.expval(qml.PauliZ(i)) for i in range(n_qubits)]
```

```
def zz_feature_map(features):
    n_wires = len(features)

    for i in range(n_wires):
        qml.Hadamard(wires=i)

    for i in range(n_wires):
        qml.RZ(features[i], wires=i)

    for i in range(n_wires):
        for j in range(i + 1, n_wires):
            angle = (np.pi - features[i]) * \
                    (np.pi - features[j])
            qml.MultiRZ(angle, wires=[i, j])
```

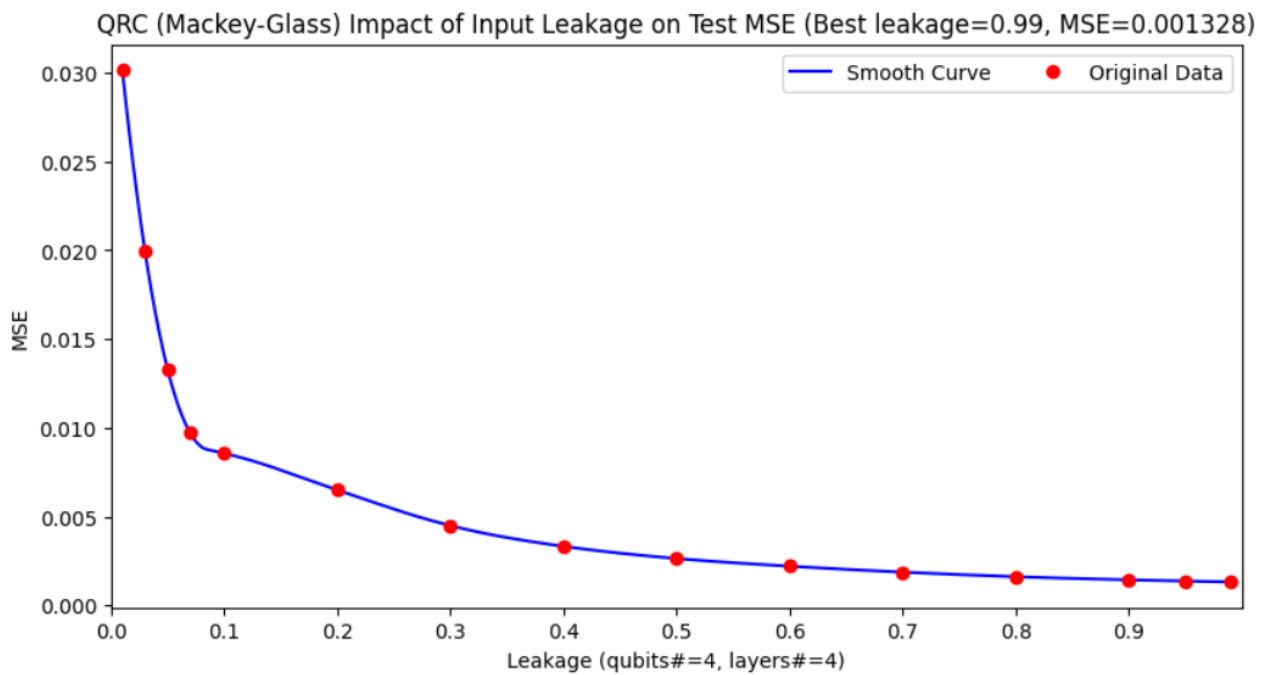


The benefit of this model is that it represents window element in phase and correlates them with higher order entanglements. This effectively spreads highly similar input values in a non-linear fashion across the Hilbert space. This means that different windows are not clustered in the Hilbert space, and the measurement outcomes can be clearly differentiated. This means that the QRC output provides more information to be combined with the reservoir and relied on in model training.

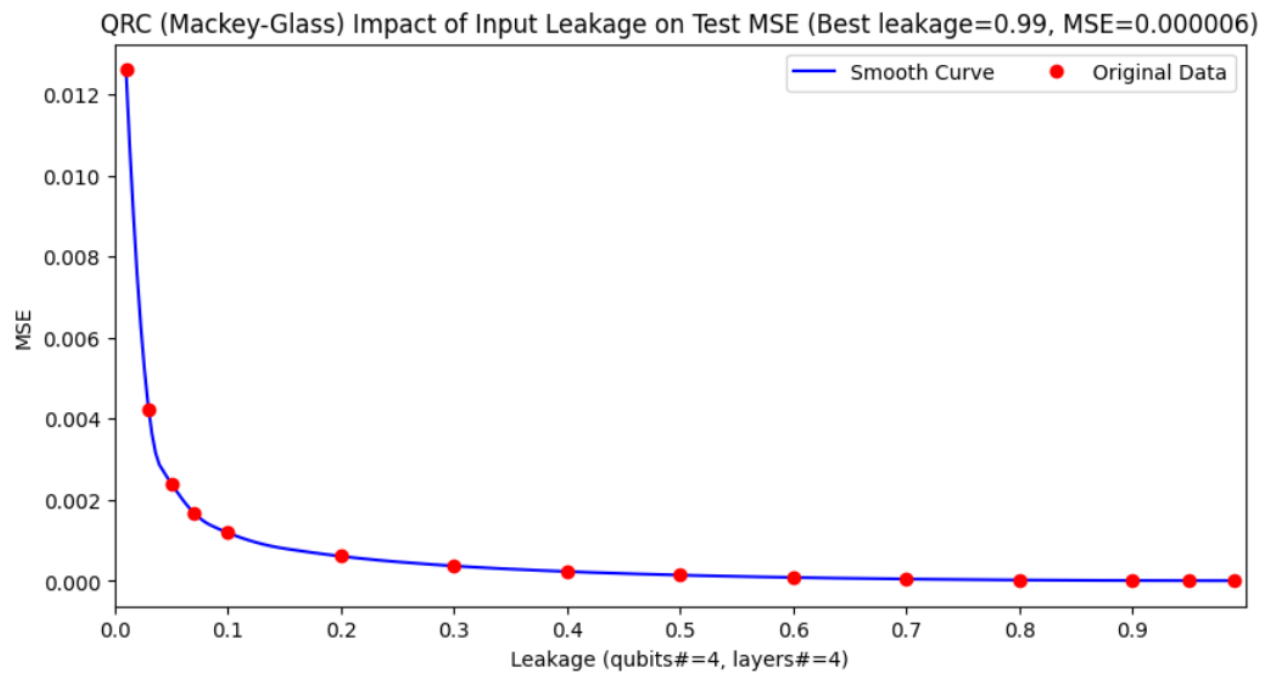
What is not clear, and should be investigated in the future, is why the model trained with long (reservoir) and short (windows) term memories together, is underperforming as compared with the model using the short-term (windows) memory only, when confronted with highly chaotic data.

In the following pages we can compare the performance of both...

The new model on Mackey-Glass data:



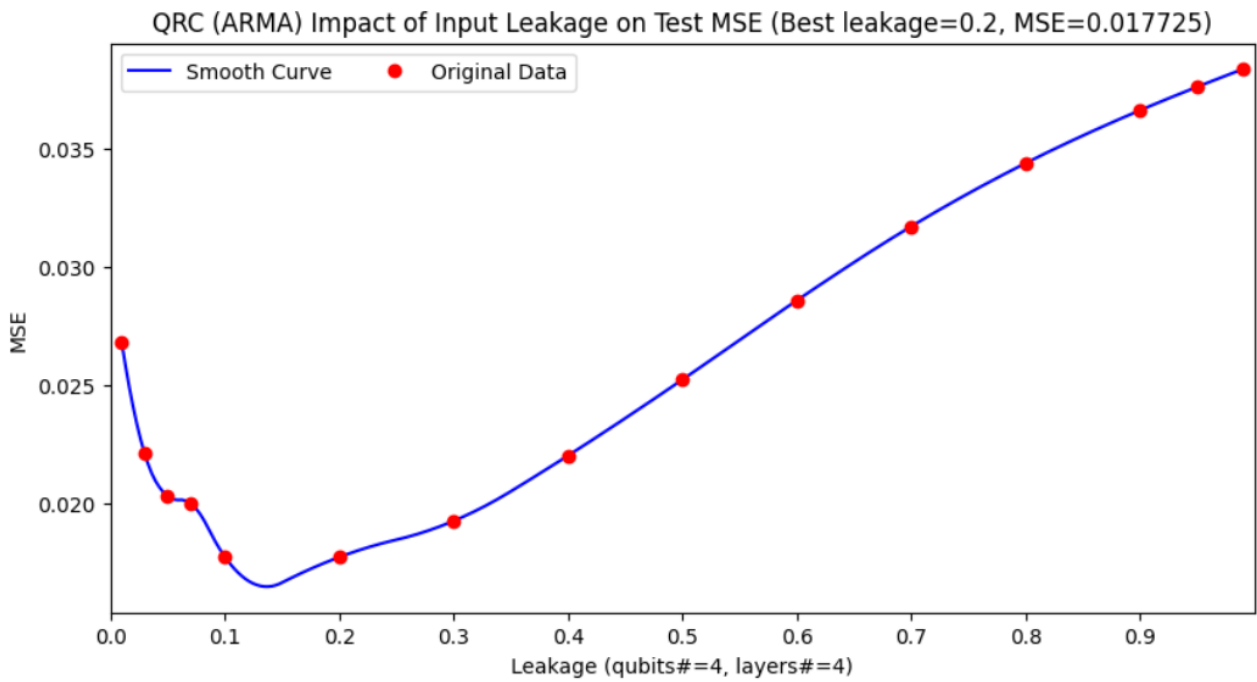
The old model on Mackey-Glass data:



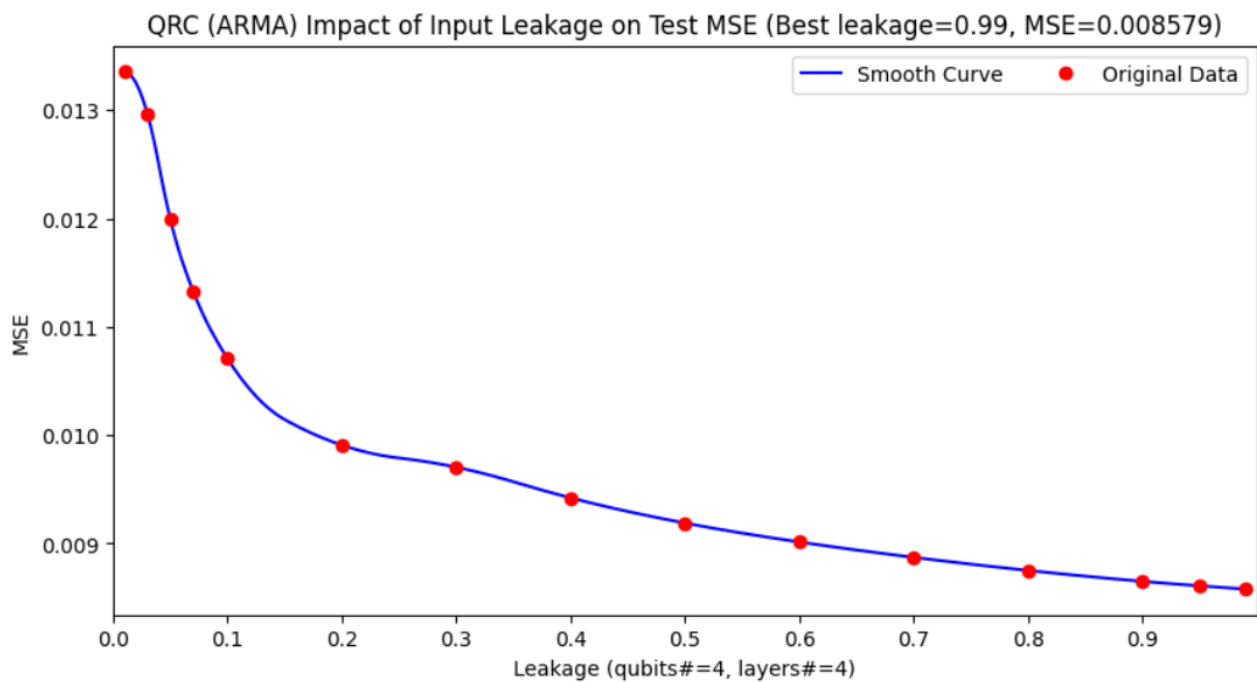
Comments:

When both models are applied to Mackie-Glass data, it is not evident that either of these two takes advantage of the dual memory of the QRC. It is evident that the old model is able to achieve a much lower MSE on test data.

The new model on ARMA data:



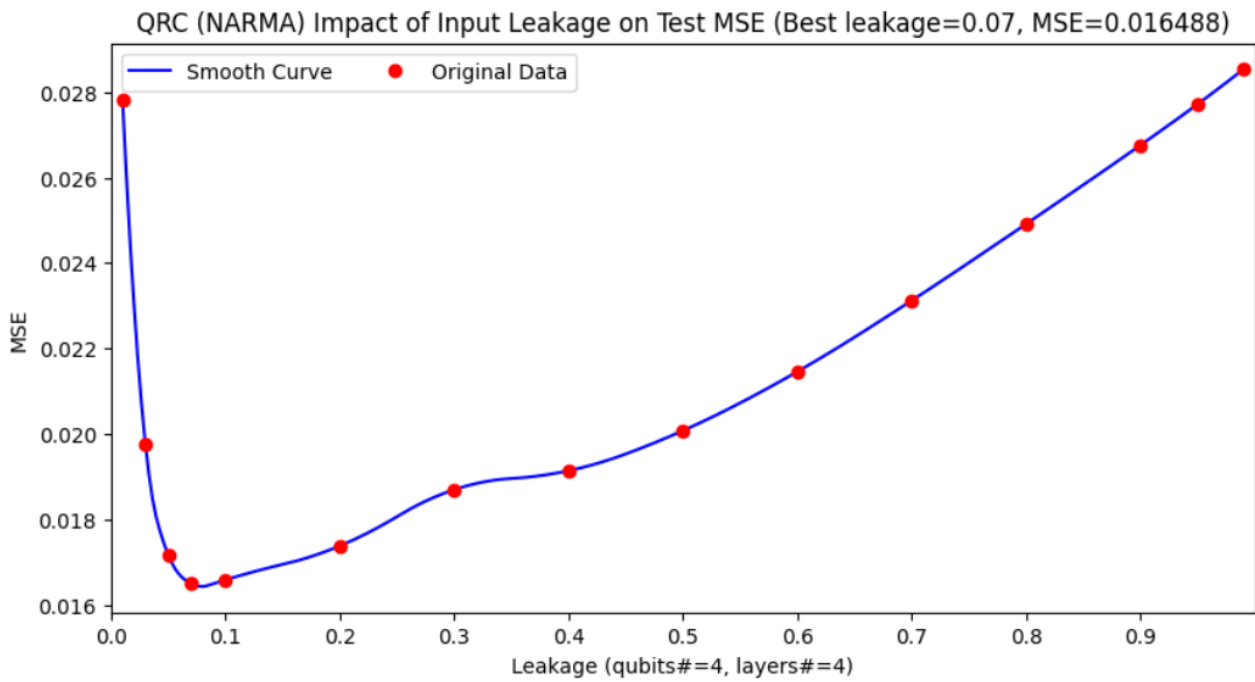
The old model on ARMA data:



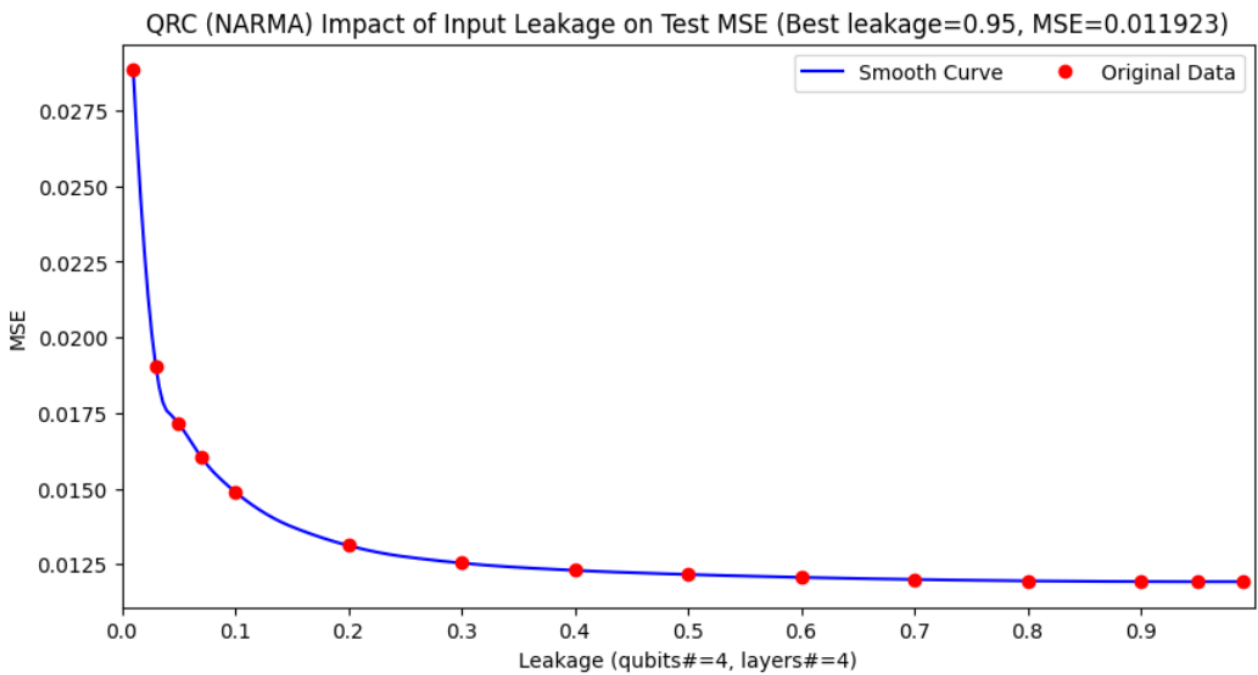
Comments:

When applied both models to ARMA data, in contrast to the old model, the new model clearly relies on the QRC dual memory as it identifies the optimum leakage rate at 0.2. The old model does not seem to rely on the reservoir at all as it progresses to take advantage of the short-term memory (window) only. In doing so, it reaches a higher MSE test score than the new model.

The new model on NARMA data:



The old model on NARMA data:



Comments:

The same situation occurs when the models are applied to NARMA data. Again, the new model clearly relies on the QRC dual memory as it identifies the optimum leakage rate at 0.07. The old model does not seem to rely on the reservoir at all as it progresses to take advantage of the short-term memory (window) only. In doing so, it reaches a higher MSE test score than the new model.